# CSS3 | Elemente positionieren

# CSS3 | position

# CSS3 | Positionierung von Objekten

Bis jetzt waren Sie es noch gewohnt, dass die HTML-Elemente im Fluss des HTML-Dokuments positioniert und angezeigt werden. Die Elemente werden dabei nacheinander in der Reihenfolge dargestellt, wie sie im HTML-Code notiert wurden.
Sie sind aber nicht auf eine solche statische Positionierung beschränkt und können dies mit CSS beeinflussen. Zum Positionieren mit CSS gibt es mehrere Möglichkeiten.

- **Positionierungs-Modell**: Positionieren von HTML-Elementen mit der CSS-Eigenschaft **position**
- **Übereinanderstapeln** von HTML-Elementen mit **z-index** regeln
- **CSS float Modul**: schwebende HTML-Elemente mit der CSS-Eigenschaft **float**
- **CSS flexbox Modul**: Positionierung von HTML-Elementen mit den neuen flexiblen Boxen aus CSS3 **display: flex**
- **CSS Grid**: **echtes** Gridmodell mit CSS3 | **display: grid;**

# CSS3 | Positionierung von Objekten

Bevor Sie die **relative**, **absolute** und **fixe** Positionierung kennenlernen, muss noch kurz auf die CSS-Eigenschaften **top, right, bottom und left** (auch **Versatzeigenschaften** genannt) eingegangen werden. Diese vier CSS-Eigenschaften werden nämlich zusätzlich für die CSS-Eigenschaft position verwendet.

**Damit geben Sie an, ob ein absolut, relativ, fixed oder sticky positioniertes Element oben (top), rechts (right), unten (bottom) und / oder links (left) beginnt**.
**Gängige Einheiten für die Werte hierfür sind Pixel (px), Prozent (%) und rem oder em.**

Von den Werten **top** und **bottom bzw**. **left** und **right** sollten Sie sinnvollerweise immer **jeweils eine Eigenschaft** verwenden. Angaben von oben und unten bzw. links und rechts sind in der Praxis eher selten sinnvoll, und eine zweite Angabe wird ignoriert, wenn diese nicht dem negativen Wert des ersten Werts entspricht.
**Negative Werte sind erlaubt.**
Der Standardwert aller vier CSS-Eigenschaften ist gleich auto, mit ihm wird die Position der entsprechenden Kanten automatisch bzw. durch seine Inhalte bestimmt.

# CSS3 | position: static;

# CSS3 | position: static;

**Positionierung mit der CSS-Eigenschaft »position«**
Wie und wo Sie ein Element positionieren und was mit den dahinter folgenden Elementen geschehen soll, legen mit der CSS-Eigenschaft **position** fest.

**Hierbei wird zwischen fünf möglichen Methoden für die Positionierung von Elementen unterschieden:**

1. **Statische Positionierung (position: static;): Das ist die Standardeinstellung für alle Elemente und wird auch verwendet, wenn Sie die CSS-Eigenschaft position gar nicht notiert haben.**
   Hierbei werden alle Elemente wie gewohnt nacheinander als Blockelemente- oder Inlineelemente angeordnet, so wie diese im HTML-Dokument notiert wurden.

# CSS3 | position: relative;

# CSS3 | position: relative;

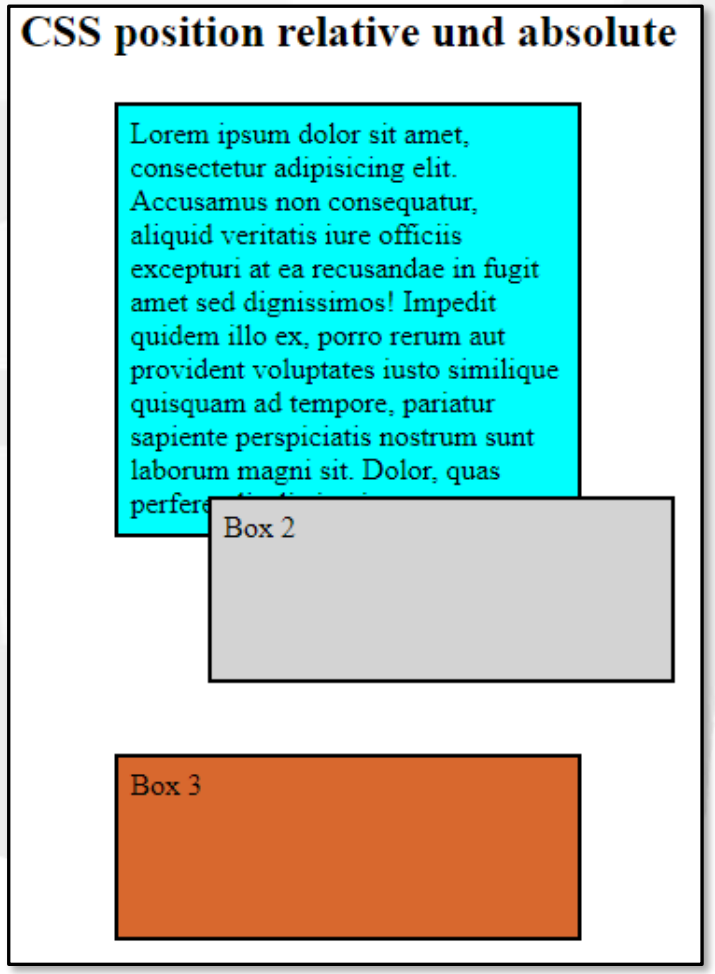
2. **Relative Positionierung** (**position: relative;**): Damit können Sie ein Element **relativ** zur aktuellen Position mit den CSS-Eigenschaften **top, bottom, left** und **right** und entsprechenden Werteangaben platzieren bzw. verschieben.
   **Die anderen Elemente werden nicht beeinflusst**:
   Die anderen Elemente bleiben an der derselben Position, als würde sich das verschobene Element immer noch an derselben Stelle befinden.

   **Bei der relativen Positionierung wird das Element von der aktuellen Position des Elements selbst verschoben. Alle anderen Elemente werden bei einer solchen Verschiebung nicht beeinträchtigt und bleiben an der Stelle, als würde das verschobene Element immer noch an derselben Stelle sein.**

   **Da die anderen Elemente nicht beeinflusst werden, können somit relativ platzierte Elemente auch andere Elemente überlagern.**

# CSS3 | position: relative;

```html
<h2>CSS position relative und absolute</h2>
<div class="box1">
    <p>Lorem ipsum dolor sit amet, consectetu
    aliquid veritatis iure officiis excepturi
    dignissimos! Impedit quidem illo ex, porr
    similique quisquam ad tempore, pariatur s
    magni sit. Dolor, quas perferendis distir
</div>
<div class="box2">
    <p>Box 2</p>
</div>
<div class="box3">
    <p>Box 3</p>
</div>
```




```css
.box1, .box2, .box3{
    width: 250px;
    height: 100px;
    border: 2px solid black;
    margin: 0.5rem auto;
    padding: 0.4rem;
}
.box1{
    background-color: aqua;
    height: auto;
    margin-top: 1.5rem;
}
.box2{
    background-color: lightgrey;
    position: relative;
    /* position: absolute; */
    top: -30px;
    left: 50px;
}
.box3{
    background-color: chocolate;
}
```

# CSS3 | position: absolute;

# CSS3 | position: absolute;

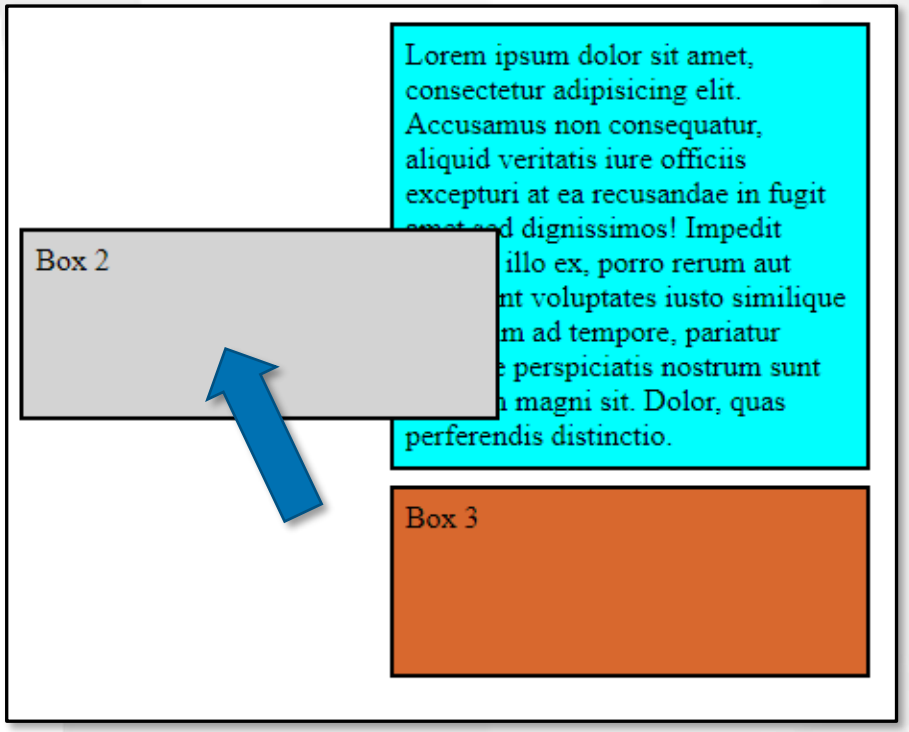
3. **Die absolute Positionierung (»position: absolute«)**
   **Bei der Verwendung der absoluten Positionierung wird das Element aus dem gewöhnlichen Dokumentenfluss herausgezogen.**

   Im Gegensatz zur relativen Positionierung reagieren jetzt alle anderen Elemente so, als wäre das absolut verschobene Element gar nicht vorhanden. Daher bleibt auch keine Lücke zwischen dem verschobenen und den nachfolgenden Elementen mehr bestehen, wie dies zuvor mit position: relative; der Fall gewesen war.
   **Auch hierbei wird die Position mit top, bottom, left und right festgelegt.**

   Die absolute Position ist hier die linke obere Ecke des Eltern-Elements, das absolut, fixiert oder relativ positioniert sein kann. Wenn kein Eltern-Element existiert, dann beziehen sich die Positionsangaben relativ zum obersten Element im Dokumentenbaum. In unserem Layout die linke obere Ecke des Browserfensters (**Viewport**) – was somit <html> wäre..

# CSS3 | position: absolute;

```html
<h2>CSS position relative und absolute</h2>
<div class="box1">
    <p>Lorem ipsum dolor sit amet, consectetu
    aliquid veritatis iure officiis excepturi
    dignissimos! Impedit quidem illo ex, porr
    similique quisquam ad tempore, pariatur s
    magni sit. Dolor, quas perferendis distir
</div>
<div class="box2">
    <p>Box 2</p>
</div>
<div class="box3">
    <p>Box 3</p>
</div>
```



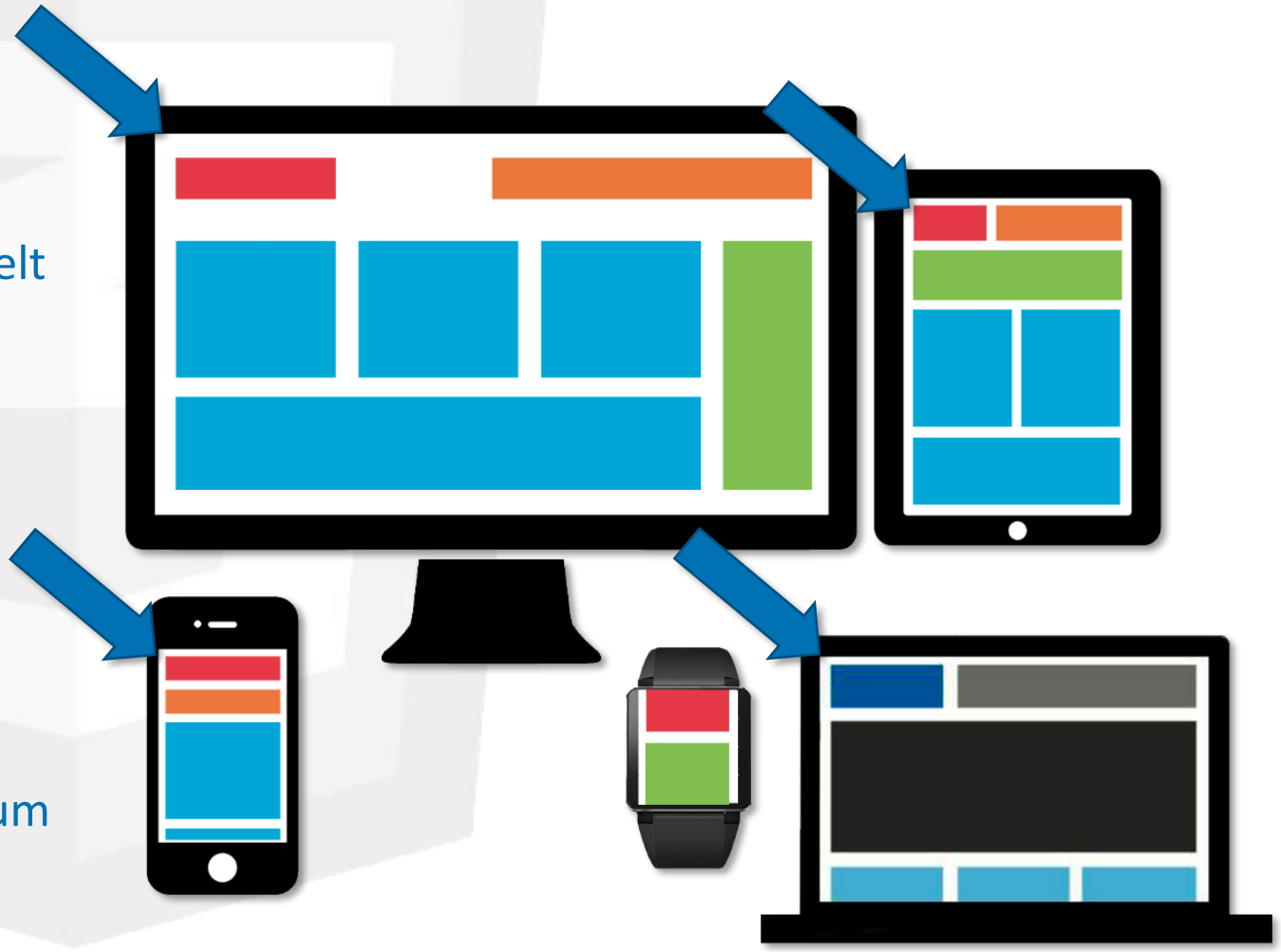

```css
.box1, .box2, .box3{
    width: 250px;
    height: 100px;
    border: 2px solid black;
    margin: 0.5rem auto;
    padding: 0.4rem;
}
.box1{
    background-color: aqua;
    height: auto;
    margin-top: 1.5rem;
}
.box2{
    background-color: lightgrey;
    /* position: relative; */
    position: absolute;
    top: -30px;
    left: 50px;
}
.box3{
    background-color: chocolate;
}
```

**Bei der Verwendung der absoluten Positionierung wird das Element aus dem gewöhnlichen Dokumentenfluss herausgezogen. Auch hierbei wird die Position mit top, bottom, left und right festgelegt.**

# CSS3 | Viewport?

**Was ist der Viewport?**
**Der Viewport** ist ein Begriff aus dem Bereich des Webdesigns und **bezieht sich auf den sichtbaren Bereich einer Webseite auf einem Gerät.** Es handelt sich um den Teil des Bildschirms, der tatsächlich angezeigt wird, ohne dass der Benutzer scrollen muss.

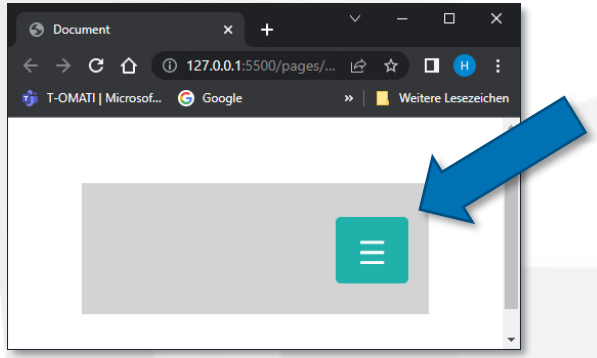
Der Viewport kann je nach Gerät variieren. Auf einem Desktop-Computer ist der Viewport normalerweise größer als auf einem Smartphone oder Tablet. Das bedeutet, dass eine Webseite für verschiedene Viewports optimiert werden sollte, um sicherzustellen, dass sie auf allen Geräten gut aussieht und benutzerfreundlich ist.



# CSS3 | position: relative; & position: absolute;

## CSS-Positionierung | position: relative; und position: absolut; arbeiten sehr gut zusammen

**Aufgabe:**
**Ein „Hamburger“ Button soll unabhängig von der Viewportgröße immer an der gleichen Stelle im Header gehalten werden.**




**☰**

**Wo finde ich die „berühmten“ drei Striche des „Hamburger“ Menüs? Siehe:**

https://de.wikipedia.org/wiki/Hamburger-Men%C3%BC-Icon#:~:text=Die%20Unicode%2DZeichen%20%E2%98%B0%20(U,(Punktreihe)%20%E2%81%9D%20minimiert%20werden.

# CSS3 | position: relative; & position: absolute;

**Aufgabe:**
**Ein „Hamburger“ Button soll unabhängig von der Viewportgröße immer an der gleichen Stelle im Header gehalten werden.**

**HTML – sehr übersichtlich ☺**

```
<header>
    <div class="menue-schalter"><a href="#">&#x2630;</a></div>
    <!-- &#x2630; ist der Unicode für einen Hamburger Menü Button -->
</header>
```

# CSS3 | position: relative; & position: absolute;

**CSS-Positionierung | position: relative; und position: absolut; arbeiten sehr gut zusammen**

**Aufgabe:**
**Ein „Hamburger“ Button soll unabhängig von der Viewportgröße immer an der gleichen Stelle im Header gehalten werden.**

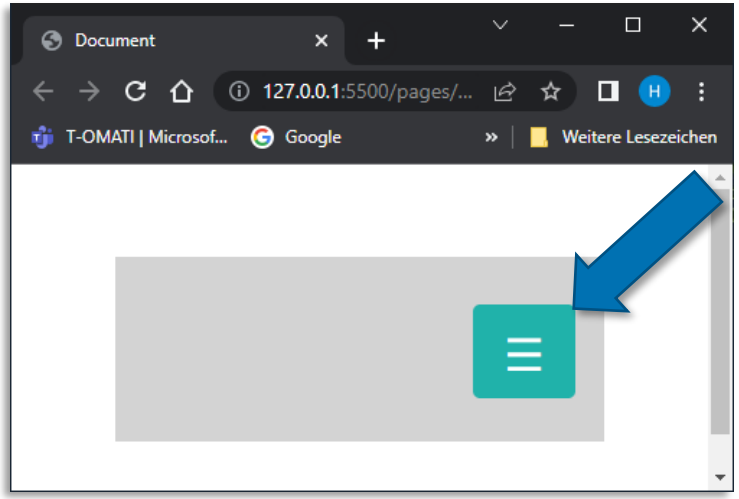
**CSS – etwas mehr ☺**

- **CSS reset nicht vergessen**

```
header{
    width: 70%;
    height: 8rem;
    background: lightgray
    margin: 4rem auto;
    position: relative;
}
.menue-schalter a{
    text-decoration: none;
    color: white;
    background-color: lightseagreen;
    padding: 0.7rem 1.3rem;
    font-size: 2rem;
    border-radius: 5px;
    position: absolute;
    bottom: 30px;
    right: 20px;
}
```

- **position: relative; hier wir nur die Positionierungseigenschaft bestimmt, ohne Positionsangaben.**

- **Position: absolute; hält den Menü Button in seinen übergeordneten Block (grau ☺).**



# CSS3 | position: fixed;

# CSS3 | position: fixed;

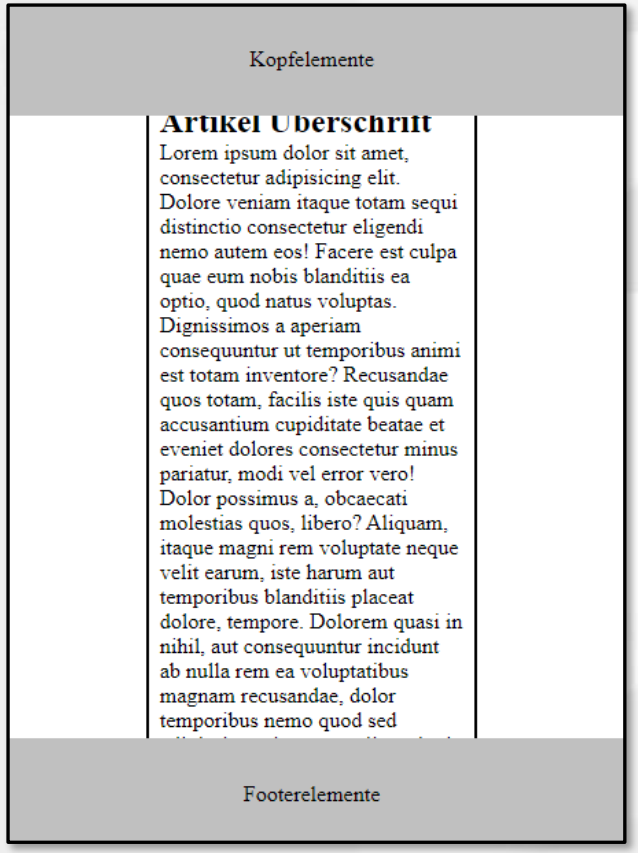
4. **Die fixe Positionierung (»position: fixed«)**
   Die fixe Positionierung entspricht dem Prinzip der absoluten Positionierung mit dem Wert absolute aus dem Abschnitt zuvor, nur mit dem **Unterschied, dass hier ein Element fixiert und somit nicht mehr mitgescrollt wird.**

   **Man richtet damit praktisch das Element starr an das Webbrowser-Fenster (auch Viewport genannt) aus.**

   In der Praxis eignet sich eine solche fixe Positionierung z. B. für Navigationsbereiche, eine fixe Kopf- oder Fußzeile oder einen fixierten Link zum Seitenfang, der immer an derselben Position vorhanden ist.

# CSS3 | position: fixed;

```html
<header>Kopfelemente</header>
<main>
    <article>…
    </article>
</main>
<footer>Footerelemente</footer>
```



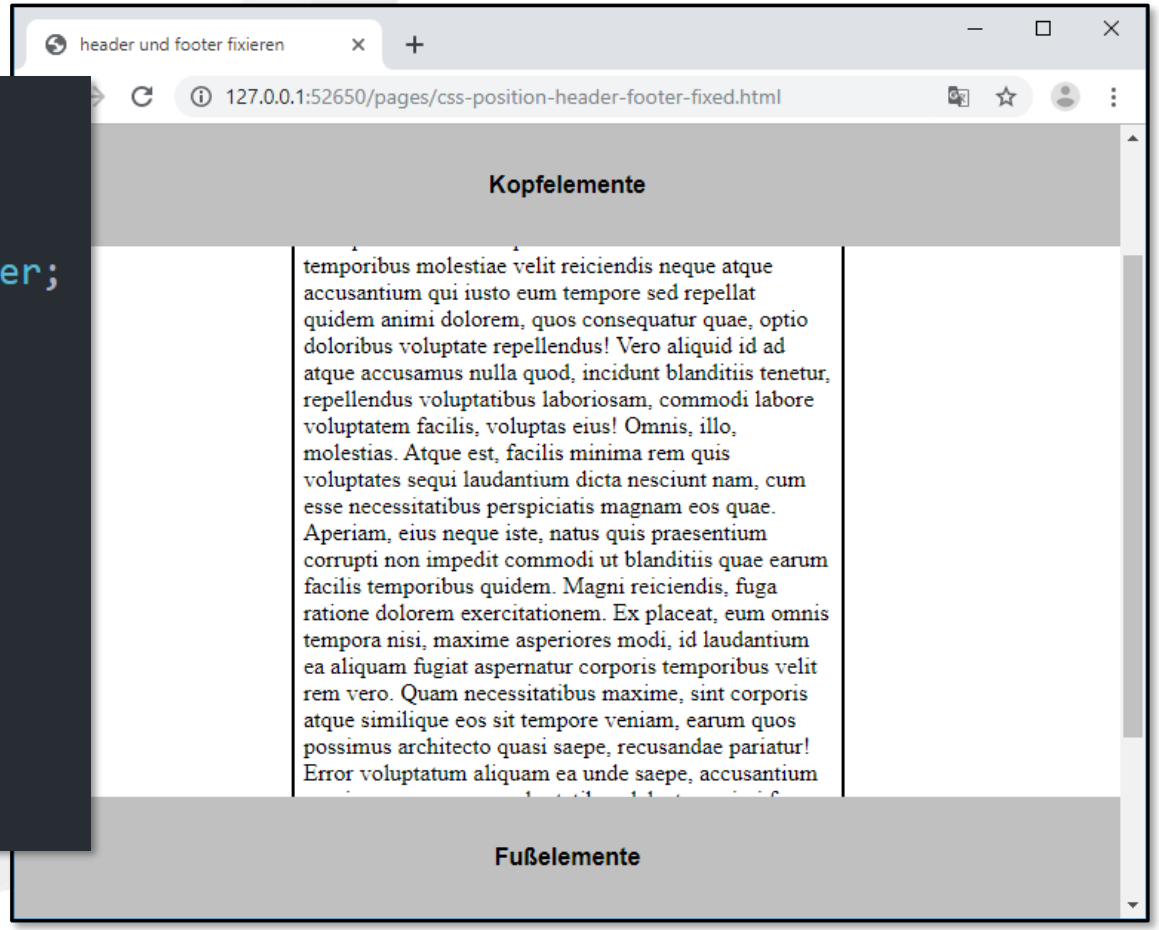

```css
header, footer{
    padding-top: 2rem;
    padding-bottom: 2rem;
    background-color: silver;
    width: 100%;
    text-align: center;
    position: fixed;
}
header{
    top: 0px;
}
footer{
    bottom: 0;
}
```

**Header und Footer sind mit position: fixed; jeweils an der oberen (top: 0px) oder unteren Grenze (bottom: 0px) des Viewports fixiert.**

# CSS3 | position: fixed;

In der Praxis eignet sich eine solche fixe Positionierung z. B. für Navigationsbereiche, eine fixe Kopf- oder Fußzeile oder einen fixierten Link zum Seitenfang, der immer an derselben Position vorhanden ist.

```
header, footer{
    padding-top: 2rem;
    padding-bottom: 2rem;
    background-color: silver;
    width: 100%;
    text-align: center;
    position: fixed;
}
header{
    top: 0px;
}
footer{
    bottom: 0;
}
```

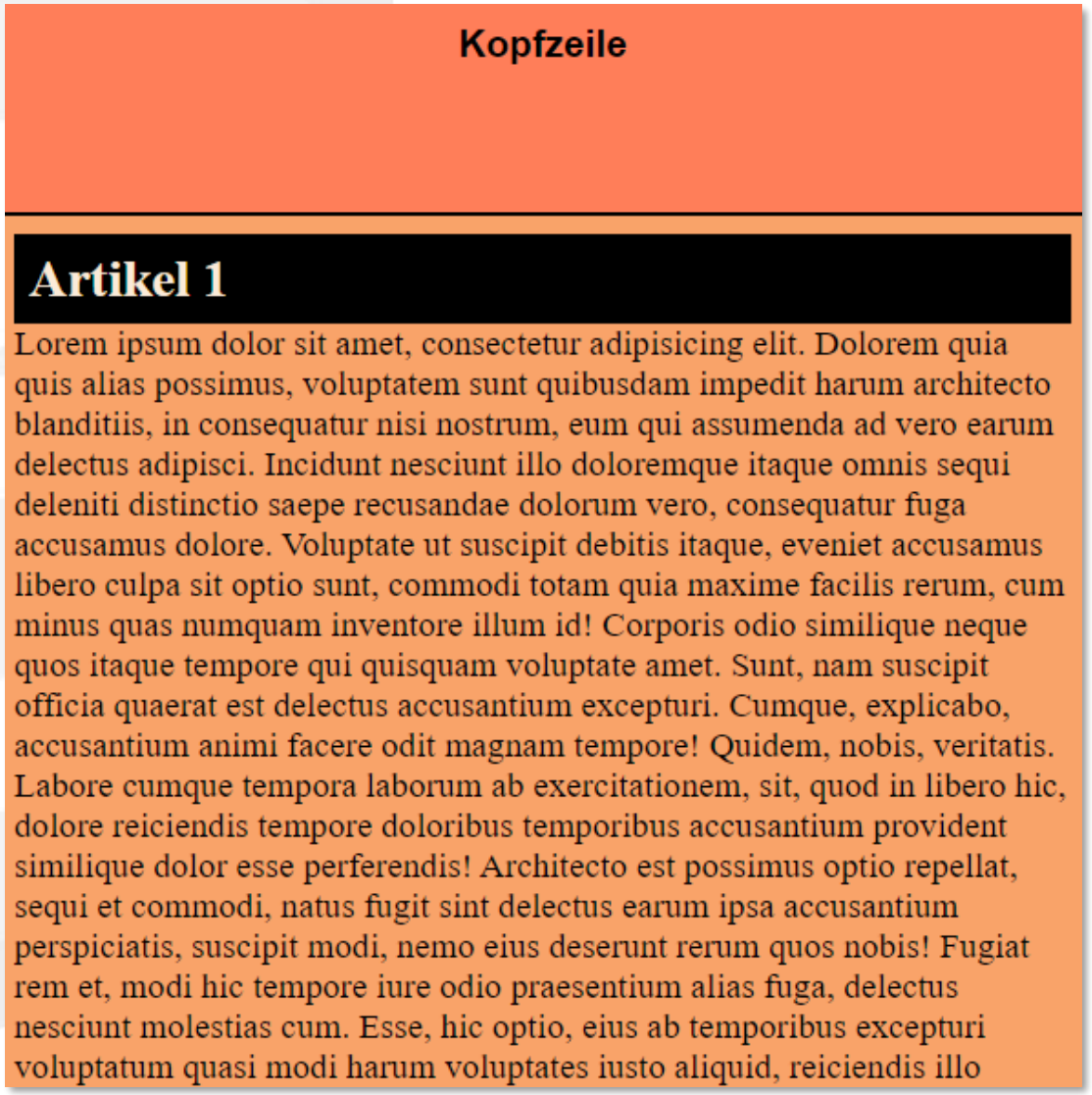


# CSS3 | position: sticky;

# CSS3 | position: sticky;

5. **Die haftende Positionierung (sticky):**
   Die haftende Positionierung ist eine Mischung aus relativen und fixen Positionierungen. Wenn Sie die Website laden, verhält sich das Element wie ein gewöhnliches HTML5 Element.

   **Wenn Sie** allerdings damit **beim Hoch- oder Herunterscrollen auf den Bildschirmrand stoßen, bleibt das Element mit position: sticky; dort haften** und verhält sich dann wie bei der fixen Positionierung.



# CSS3 | position: sticky;

Ich habe die Klassen **sticky-art1** und **sticky-art2** für die Artikelüberschriften im HTML-Dokument erstellt. Dabei wurde der CSS-Eigenschaft **position** der Wert **sticky** übergeben.

Die eigentliche Positionierung (hier **top: 10px**) gilt dann, wenn der obere Bildschirmrand beim Hochscrollen erreicht wird.

Wenn Sie die Artikel hochscrollen, bleibt die Überschrift am oberen Bildschirmrand haften.

HTML5

```html
<header>Kopfzeile</header>
<main>
  <section>
    <article class="art1">
      <h2 class="sticky-art1">Artikel 1</h2>
      <p>Lorem ipsum dolor sit amet, consectetur adipisicing
```

CSS3

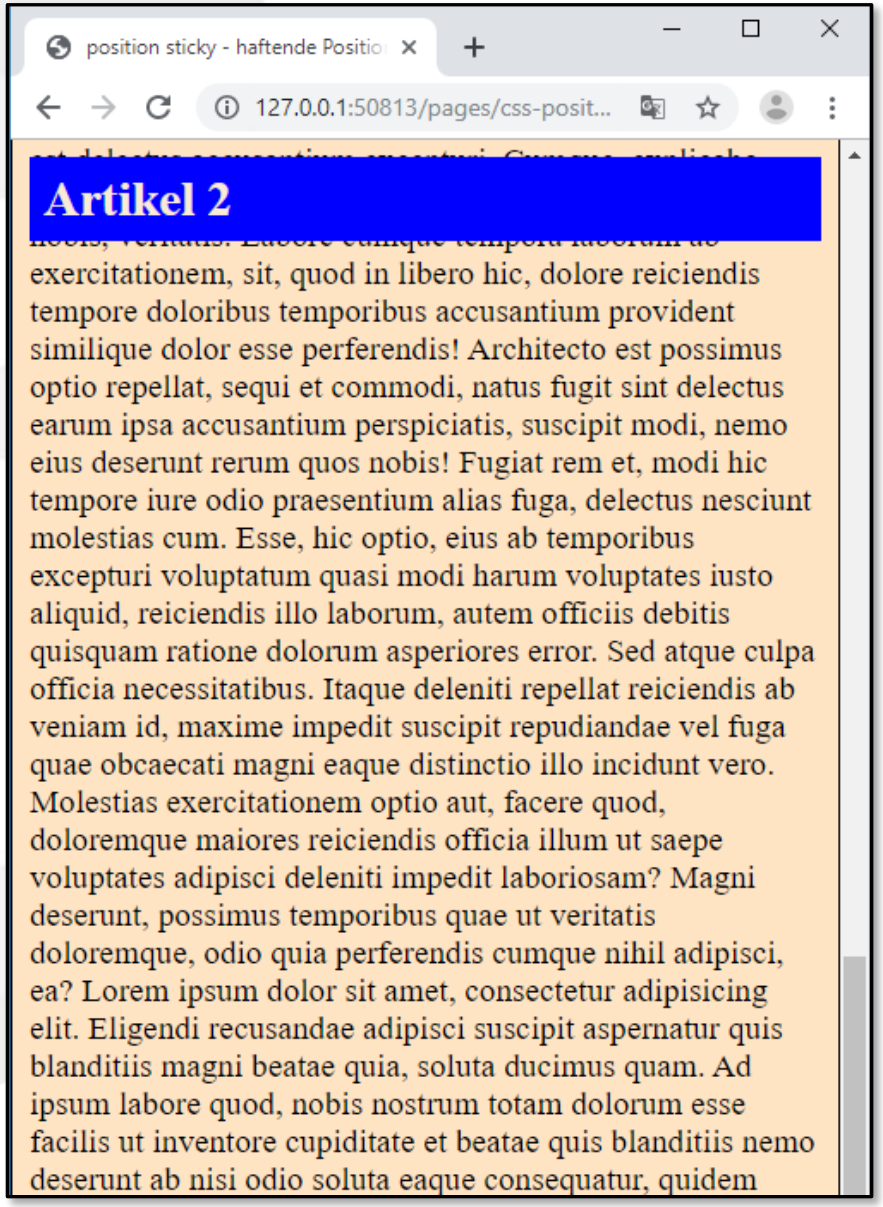
```css
.sticky-art1, .sticky-art2{
  position: sticky;
  top: 10px;
  width: 100%;
  background: black;
  color: antiquewhite;
  padding: 0.5rem;
}
.sticky-art2{
  background: blue;
}
```

# CSS3 | position: sticky;

Dies geht so lange, bis Sie auf die nächste Überschrift stoßen, womit diese dann wiederum oben haften bleibt.

Hiermit haben Sie immer die Überschrift des aktuellen Artikels im Blick.

Scrollen Sie die Website wieder ganz nach oben, löst sich diese Haftung wieder.



# CSS3 | z-index: …;

# CSS3 | z-index: …;

**Übereinanderstapeln regeln mit »z-index«**
In den Beispielen mit der relativen und absoluten Positionierung zuvor haben Sie bereits gesehen, dass, wenn Sie Elemente absolut (oder auch relativ oder fixiert) in ihrer Position verändern oder aus dem Dokumentenfluss herausnehmen, es zu Überlappungen der Elemente kommen kann. **Standardmäßig liegen die relativen, absoluten und fixierten Elemente in der Reihenfolge aufeinandergestapelt, wie sie im Dokumentenfluss des HTML-Dokuments notiert wurden. Das zuletzt notierte Element liegt oben.**

**Dieses Verhalten können Sie mit der CSS-Eigenschaft z-index verändern.**

**Die CSS-Eigenschaft z-index ist quasi die dritte Achse bei der Positionierung eines Elementes.**
Bisher haben Sie sich immer nur um die horizontale und vertikale Position gekümmert (also die x- und y-Achse). Die dritte Dimension ist die z-Achse, wo die Elemente übereinanderliegen, deswegen auch z-index.

**Achtung! z-index ist immer an eine position Angabe gebunden!!!**

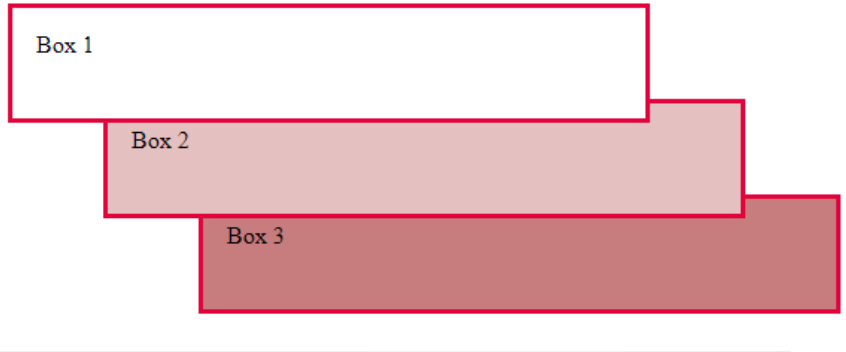
# CSS3 | z-index: ...;

```html
<body>
    <div class="eins">Box 1</div>
    <div class="zwei">Box 2</div>
    <div class="drei">Box 3</div>
</body>
```

```css
div {
    position: absolute;
    width: 75%;
    min-height: 5rem;
    padding: 1rem;
    margin: 0.5rem;
    border: 3px solid crimson;
}
```

```css
.eins {
    top: 0;
    left: 0;
    background: white;
    z-index: 30;
}
.zwei {
    top: 4rem;
    left: 4rem;
    background: #e3c0c0;
    z-index: 20;
}
.drei {
    top: 8rem;
    left: 8rem;
    background: #c47d7d;
    z-index: 10;
}
```

**Negative Werte für »z-index«**
für die CSS-Eigenschaft ?
Für **z-index** können auch **negative Werte** verwendet werden.
Natürlich gilt hier nach wie vor, dass Elemente mit positiven z-index-Werten immer über den Elementen mit negativen z-index-Werten angeordnet werden.




**Zwei Dinge sind noch wichtig:**

- **z-index gilt nur für positionierte Boxen.**
- **z-index ist nur relevant, wenn Boxen sich überlappen.**

# CSS3 | Position im responsiven Webdesign

# CSS3 | z-index: …;

**Beispiel mobile Navigation:**

- **hier der Menü Button im Mobile Layout**
- **Übrigens nur mit CSS3 realisiert – ohne JavaScript ☺**

```html
<header class="header arrow-down"
role="banner">
  <div class="logo"><a href="#">RWD-
  Beispiele</a></div>
  <a class="toggle-nav"
  href="#navMain">Menü</a>
</header>
```

```css
a.toggle-nav {
  position: absolute;
  top: 0.4rem;
  right: 0.5rem;
  display: inline-block;
  padding: 0.357em;
  font-weight: 300;
  border-radius: 0.2em;
  text-decoration: none;
  cursor: pointer;
}

a.toggle-nav:hover {
  background: #96cfbf;
```

# VIELEN DANK!